Py-02 · Strutture condizionali

Indice

1 Strutture condizionali **2**
- 1.1 Costrutto condizionale IF . . . 2
- 1.2 Esempio: maggiorenne o minorenne? . . . 2
- 1.3 IF - ELSE . . . 3
- 1.4 Esempio: maggiorenne o minorenne? . . . 4
- 1.5 IF - ELIF - ELSE . . . 4
- 1.6 Strutture nidificate (Nested if-elif-else) . . . 6
- 1.7 Strutture condizionali e operatori logici . . . 7

2 Schema generale nel quale inquadrare l'uso del costrutto condizionale **8**
- 2.1 Schemi logici delle decisioni . . . 8
- 2.2 Guardia . . . 9
- 2.3 Scelta binaria . . . 10
- 2.4 Scelta multipla vs classificazione per intervalli . . . 10
- 2.5 Scelta multipla vs tabella decisionale . . . 11
- 2.6 Decisione composta . . . 11
- 2.7 Albero decisionale . . . 12

3 Altre considerazioni **13**
- 3.1 Altre considerazioni: **Condizioni mutuamente esclusive** . . . 13
- 3.2 Altre considerazioni: **Condizioni indipendenti** . . . 14
- 3.3 Variabile *flag* . . . 15
- 3.4 Variabile contatore . . . 15
- 3.5 Equivalenza tra strutture nidificate e strutture flat . . . 16

4 Esempi ed esercizi (DRAFT) **19**
- 4.1 Esempio: autorizzazione all'accesso a un'area riservata . . . 19
- 4.2 Nidificata o flat? . . . 20
- 4.3 Esercizi . . . 20
- 4.4 Sistema di prenotazione biglietti . . . 21
- 4.5 Prestito bancario . . . 22
- 4.6 Gestione ordini . . . 22
- 4.7 Esempio: voto in base al punteggio . . . 23
- 4.8 Esempio: promosso, rimandato o bocciato . . . 23

1 Strutture condizionali

1.1 Costrutto condizionale IF

Un programma può:

- eseguire istruzioni in sequenza
- ripetere istruzioni (cicli)
- scegliere tra più azioni

A volte è necessario eseguire (o non eseguire) un dato blocco di codice a seconda che determinate condizioni siano soddisfatte. La struttura condizionale controlla l'esecuzione di un gruppo di istruzioni in base al **valore di verità** di un'espressione. In altre parole, **le strutture condizionali permettono al programma di scegliere quale codice eseguire in base a una condizione**.

L'istruzione `if` valuta la condizione `cond_0`. Se `cond_0` è `True`, viene eseguito il blocco di codice associato (ramo `True`); se è `False`, il blocco viene saltato. In entrambi i casi l'esecuzione prosegue con l'istruzione successiva alla struttura condizionale (`# Next statement`).

Sintassi minimale:

```
if cond_0:
    # True branch
    # Run if cond_0 is true
    # Your code goes here...

# Next statement
```

Un **blocco di codice** è un insieme di istruzioni che appartengono logicamente a una struttura di controllo come una funzione, un ciclo o un costrutto condizionale `if`. Il blocco di codice è spostato più a destra (indentato) rispetto all'istruzione che lo controlla. In Python l'indentazione è obbligatoria: è proprio lei a stabilire quali istruzioni appartengono al blocco.

1.2 Esempio: maggiorenne o minorenne?

```
print("quanti anni hai?")
eta = int(input())
if eta >= 18:
    print("sei maggiorenne")
```

Esempio con errore: il seguente codice *non* stampa 'sei maggiorenne' se hai esattamente 18 anni!

```
print("quanti anni hai?")
eta = int(input())
if eta > 18:
    print("sei maggiorenne")
```

! Regole sintattiche del costrutto `if`

- Dopo la condizione i **due punti** `:` sono obbligatori.
- Le istruzioni del blocco vanno **indentate**, tutte con lo stesso numero di spazi (per convenzione **4 spazi**).
- Il blocco termina alla prima istruzione che torna al livello di indentazione dell'`if`.

Se una regola non è rispettata, Python non esegue il programma e segnala un errore.

```
if eta >= 18                # mancano i due punti
    print("sei maggiorenne")
# SyntaxError: expected ':'
```

```
if eta >= 18:
print("sei maggiorenne")     # istruzione non indentata
# IndentationError: expected an indented block after 'if' statement
```

```
if eta >= 18:
    print("sei maggiorenne")
  print("puoi votare")       # 2 spazi invece di 4
# IndentationError: unindent does not match
#                   any outer indentation level
```

! Errore tipico: `=` invece di `==`

Nella condizione si usa l'operatore di confronto `==`, non l'assegnazione `=`.

```
if x = 5:      # ERRATO: = assegna un valore, non confronta
    print("x vale 5")
# SyntaxError: invalid syntax.
# Maybe you meant '==' or ':=' instead of '='?
```

```
if x == 5:     # CORRETTO: confronta x con 5
    print("x vale 5")
```

1.3 IF - ELSE

A seconda del valore della condizione, viene eseguito il ramo `True` o il ramo `False`

```
if cond_0:
    # True branch
    # Run if cond_0 is true
    # Your code goes here...
else:
```

```
    # False branch
    # Run if cond_0 is false
    # Your code goes here...

# Next statement
```

1.4 Esempio: maggiorenne o minorenne?

```
print("quanti anni hai?")
eta = int(input())
if eta >= 18:
    print("sei maggiorenne")
else:
    print("sei minorenne")
```

Se voglio che venga eseguito del codice solo se la condizione è **falsa** devo comunque definire anche un ramo `True`, che è obbligatorio.

Per avere un ramo `True` vuoto posso utilizzare l'istruzione `pass`, che indica l'istruzione vuota ('nessuna azione').

https://docs.python.org/it/3.12/reference/simple_stmts.html#the-pass-statement

```
if cond:
    pass
else:
    print("sono nel ramo False")
```

Per eseguire codice solo quando la condizione è falsa, il modo naturale è negarla.

```
if not cond:
    print("sono nel ramo True, la condizione 'cond' è falsa")
```

L'istruzione `pass` resta utile come 'segnaposto', nel caso, per esempio, di codice ancora da definire.

1.5 IF - ELIF - ELSE

Potrei avere più condizioni da testare 'in cascata'.

Voglio ottenere il seguente comportamento:

- Se l'età è minore di 18, viene stampato "sei minorenne"
- Se l'età è tra 18 e 64, viene stampato "sei un adulto"
- Se l'età è tra 65 e 115, viene stampato "sei un anziano"
- Se l'età è 116 o più, viene stampato "Inserimento non valido o età fuori dal range."

Controllo se l'età è minore di 18 (**prima condizione**).

- Se la condizione è vera, viene stampato "sei minorenne" ed esco dal costrutto condizionale.

- Solo se la condizione è falsa controllo la **seconda condizione**.

Controllo se l'età è < 65 (se sono in questo ramo del codice significa che la prima condizione è falsa; non ho necessità di ricontrollarla)

- Se la condizione è vera, viene stampato "sei un adulto" ed esco dal costrutto condizionale.
- Solo se la condizione è falsa controllo la **terza condizione**.

Controllo se l'età è < 116 (se sono in questo ramo del codice significa che la seconda condizione è falsa; non ho necessità di ricontrollarla)

- Se la condizione è vera, viene stampato "sei un anziano" ed esco dal costrutto condizionale.
- Se anche l'ultima condizione è falsa viene stampato "Inserimento non valido o età fuori dal range." ed esco dal costrutto condizionale.

```
print("quanti anni hai?")
eta = int(input())

if eta < 18: # ATTENZIONE: <= 18 sarebbe errato
    print("sei minorenne")
elif eta < 65:
    print("sei un adulto")
elif eta < 116:
    print("sei un anziano")
else:
    print("Inserimento non valido o età fuori dal range.")
```

Sintassi completa del costrutto `if`

Questa è la sintassi completa del costrutto `if`. Solo la parola chiave `if`, la prima condizione e il blocco `True` sono obbligatori. Tutto il resto è opzionale.

Sta a noi utilizzare questa sintassi per ottenere la corretta esecuzione del codice in base alle condizioni che vogliamo valutare.

```
# solo IF
if cond_0:
    # CODICE DA ESEGUIRE...
# Next statement

# ----------

# IF ELSE
if cond_0:
    # CODICE DA ESEGUIRE...
else:
    # CODICE ALTERNATIVO DA ESEGUIRE...
# Next statement

# ----------
```

```
# IF ELIF ELSE

if cond_0:
    # Valuta cond_0.
    # Se cond_0 è True, esegui il codice ed esci dall'IF
    #    CODICE DA ESEGUIRE...
elif cond_1:
    # Solo se cond_0 è False, valuta cond_1.
    # Se cond_1 è True, esegui il codice ed esci dall'IF
    #    CODICE DA ESEGUIRE...
elif cond_2:
    # Solo se cond_1 è False, valuta cond_2.
    # Se cond_2 è True, esegui il codice ed esci dall'IF
    #    CODICE DA ESEGUIRE...
...
else:
    # Solo se tutte le condizioni precedenti sono False,
    # esegui il codice ed esci dall'IF
    #    CODICE DA ESEGUIRE...

# Next statement
```

Le condizioni vengono valutate in cascata.

Viene eseguito il codice in corrispondenza della **prima** condizione `True`, e in seguito il costrutto **IF** termina (il programma prosegue alla linea `# Next statement`).

Se tutte le condizioni sono `False`, viene eseguito il codice associato alla clausola `else` (facoltativa), se esiste. Se le condizioni sono `False` e non esiste la clausola `else`, nessun ramo viene eseguito.

In ogni caso viene eseguito al più un **blocco di codice**.

1.6 Strutture nidificate (Nested if-elif-else)

Nei rami del costrutto condizionale (`if`, `elif`, ..., `else`) possono essere presenti altri costrutti condizionali. Questo significa che possiamo scrivere un altro `if-elif-else` all'interno dei rami del costrutto.

```
if cond_0:
    if cond_10:
        #    CODICE DA ESEGUIRE...
    elif cond_11:
        #    CODICE DA ESEGUIRE...
    elif cond_12:
        #    CODICE DA ESEGUIRE...
    else:
        #    CODICE DA ESEGUIRE...
elif cond_1:
    #    CODICE DA ESEGUIRE...
elif cond_2:
```

```
    #    CODICE DA ESEGUIRE...
else:
    #    CODICE DA ESEGUIRE...

# Next statement
```

- La struttura IF interna (`if cond_10:`) viene valutata solo se `cond_0` è vera.
- Ogni **`elif`** o **`else`** appartiene all'**`if`** che lo precede **allo stesso livello di indentazione**.
- **In pratica non ci sono limiti alla profondità e alla lunghezza della struttura.**

Utilizzare strutture nidificate molto complesse non è una buona idea. Il codice diventa difficile da capire e da modificare. Meglio combinare le condizioni semplici mediante operatori logici, o usare altri strumenti che vedremo in seguito.

1.7 Strutture condizionali e operatori logici

La condizione che compare nel costrutto `if` può essere ottenuta combinando varie condizioni mediante gli **operatori logici (booleani)** `and`, `or` e `not` (vedere la relativa sezione).

- `condizione = cond_1 or cond_2` -> `True` se almeno una condizione è vera
- `condizione = cond_1 and cond_2`
- `condizione = cond_1 or not cond_2`
- `condizione = cond_1 or (cond_2 and cond_3 )`
- `condizione = not (cond_1 or cond_2) and (cond_1 or cond_3)`

Esempio: Controllare se un utente può accedere a una risorsa in base a età e abbonamento.

```
eta = 20
abbonato = True

if ((eta >= 18) and abbonato): # parentesi superflue, le metto per
 ↪  chiarezza
    print("Accesso consentito.")
else:
    print("Accesso negato.")
```

L'uso di `and`, `or` e `not` consente di creare logiche complesse in modo chiaro ed efficace.

Strutturando bene le condizioni si possono evitare `if` nidificati non necessari e il codice resta leggibile.

! Confronti concatenati

Per verificare se un valore è compreso in un intervallo, Python permette di scrivere i confronti **concatenati**, come in matematica:

```
if 18 <= eta < 65:              # equivale a: 18 <= eta and eta < 65
    print("sei un adulto")
```

Sono comodi quando gli intervalli vanno scritti in modo esplicito, senza dipendere dall'ordine dei rami:

```
if 0 <= eta < 18:
    print("sei minorenne")
elif 18 <= eta < 65:
    print("sei un adulto")
elif 65 <= eta < 116:
    print("sei un anziano")
else:
    print("Inserimento non valido o età fuori dal range.")
```

A differenza della versione precedente, un'età negativa viene segnalata come non valida. Usarli solo nella forma "ordinata" `a < x < b` (o con `<=`): espressioni come `a < x > b` sono lecite ma poco chiare.

2 Schema generale nel quale inquadrare l'uso del costrutto condizionale

2.1 Schemi logici delle decisioni

- Quando si utilizzano strutture condizionali per risolvere un problema è utile ragionare prima sulla **logica della decisione**, indipendentemente dal linguaggio di programmazione. Molti problemi apparentemente diversi possono infatti essere ricondotti a pochi **schemi decisionali ricorrenti**. Riconoscere questi schemi aiuta a capire come strutturare il ragionamento prima ancora di scrivere il codice.
- Un primo schema è la **guardia**. In questo caso la decisione serve a verificare se le condizioni minime del problema sono soddisfatte prima di proseguire. Non si tratta ancora di scegliere tra diverse azioni, ma di stabilire se l'operazione ha senso oppure no. La guardia separa i casi validi da quelli invalidi e serve spesso a gestire input errati o situazioni eccezionali. Dal punto di vista logico il problema è quindi diviso in due parti: i casi in cui si può procedere e quelli in cui bisogna fermarsi o segnalare un errore.
- Un secondo schema è la **scelta binaria**. Qui il problema ha due possibili esiti alternativi e la decisione consiste nel determinare a quale dei due casi appartiene la situazione considerata. La condizione diventa quindi un criterio che separa il dominio in due insiemi: se la proprietà è soddisfatta si segue un comportamento, altrimenti se ne segue un altro. Questo schema è il modo più semplice di usare una decisione per indirizzare il comportamento di un programma.
- Un'estensione naturale della scelta binaria è la **scelta multipla**. In molti problemi non esistono solo due casi ma diverse alternative possibili. La decisione consiste allora nell'identificare quale categoria descrive correttamente la situazione corrente. Dal punto di vista logico è importante che le categorie siano ben definite e possibilmente non sovrapposte, in modo che ogni situazione appartenga chiaramente a uno dei casi previsti.
- Un caso molto frequente della scelta multipla è la **classificazione per intervalli**. Qui il valore di una grandezza numerica determina a quale fascia appartiene la situazione. Il dominio viene quindi suddiviso in intervalli ordinati, ognuno associato a un risultato o a un comportamento. Questo schema richiede particolare attenzione ai limiti degli

intervalli, perché bisogna stabilire con precisione dove inizia e dove termina ciascuna fascia e assicurarsi che tutte le possibilità siano coperte.

- Un altro schema importante è la **decisione composta**. In questo caso il risultato dipende dalla **combinazione di più condizioni**. Non basta verificare una singola proprietà: la decisione corretta emerge dalla relazione tra diversi fattori. Dal punto di vista logico il problema consiste nel capire quali condizioni devono essere soddisfatte contemporaneamente e quali invece rappresentano alternative possibili.
- In alcuni problemi le decisioni non sono sullo stesso livello ma si organizzano in una sequenza. Questo porta allo schema dell'**albero decisionale**. Una prima decisione determina quale seconda decisione ha senso prendere, e così via. Il ragionamento si sviluppa quindi come una serie di domande successive che restringono progressivamente il numero dei casi possibili. Questo schema è utile quando il problema può essere scomposto in passi decisionali più semplici.
- Infine, quando il risultato dipende da diverse combinazioni di condizioni e il numero dei casi è limitato, può essere utile ragionare in termini di **tabella decisionale**. In questo approccio si elencano sistematicamente tutte le combinazioni rilevanti di condizioni e il risultato associato a ciascuna di esse. La tabella permette di verificare che tutti i casi siano stati considerati e che non ci siano sovrapposizioni o omissioni.

Schema riassuntivo

Schema logico	Idea centrale
Guardia	verificare che il problema sia valido prima di procedere
Scelta binaria	decidere tra due possibili alternative
Scelta multipla	identificare quale categoria descrive il caso
Classificazione per intervalli	assegnare un valore a una fascia numerica
Decisione composta	il risultato dipende da più condizioni
Albero decisionale	una decisione determina quale altra decisione prendere
Tabella decisionale	il risultato dipende da combinazioni finite di condizioni

2.2 Guardia

Controllo preliminare per verificare se il problema è valido.

```
if x < 0:
    print("errore")
# oppure

if b != 0:
    c = a/b
```

2.3 Scelta binaria

Due possibili alternative.

```
if n % 2 == 0:
    print("pari")
else:
    print("dispari")
```

2.4 Scelta multipla vs classificazione per intervalli

Scelta multipla e classificazione per intervalli utilizzano la stessa struttura di codice (`if-elif-else`), ma il problema è leggermente diverso dal punto di vista logico:

2.4.1 Scelta multipla

Le categorie non derivano da un ordine numerico. Sono casi discreti e mutuamente esclusivi.

```
if op == "+":
    r = a + b
elif op == "-":
    r = a - b
elif op == "*":
    r = a * b
else:
    print("operatore non valido")
```

2.4.2 Classificazione per intervalli

Le categorie derivano da un ordine numerico e da soglie.

```
if voto < 18:
    print("insufficiente")
elif voto < 24:
    print("sufficiente")
elif voto < 28:
    print("buono")
else:
    print("ottimo")
```

2.5 Scelta multipla vs tabella decisionale

Scelta multipla e tabella decisionale utilizzano (quasi) la stessa struttura di codice, ma il problema è leggermente diverso dal punto di vista logico:

2.5.1 Scelta multipla

La decisione dipende da una sola variabile.

```
if giorno == 1:
    print("lunedì")
elif giorno == 2:
    print("martedì")
elif giorno == 3:
    print("mercoledì")
```

2.5.2 Tabella decisionale

Il risultato dipende da più variabili contemporaneamente.

```
if premium and spesa > 100:
    sconto = 20
elif premium:
    sconto = 10
elif spesa > 100:
    sconto = 5
else:
    sconto = 0
```

2.6 Decisione composta

Il risultato dipende da più condizioni.

```
if eta >= 18 and patente:
    print("puoi guidare")
```

2.7 Albero decisionale

Un albero decisionale è appropriato quando una prima decisione determina quale seconda decisione ha senso valutare. Le condizioni quindi non sono sullo stesso livello, ma organizzate gerarchicamente.

Esempio: prezzo del biglietto al cinema

Regole:

```
se lo spettatore è studente
- se ha meno di 18 anni → prezzo ridotto
- altrimenti → prezzo studente

se non è studente
- se ha meno di 18 anni → prezzo ridotto
- altrimenti → prezzo intero
```

Logica del problema

```
studente?
   sì
       età < 18 ?
           sì → ridotto
           no → studente
   no
        età < 18 ?
            sì → ridotto
            no → intero
```

```
studente = True
eta = 20

if studente:
    if eta < 18:
        prezzo = 5
    else:
        prezzo = 7
else:
    if eta < 18:
        prezzo = 5
    else:
        prezzo = 10

print(prezzo)
```

NOTA: la logica del problema è un albero, ma **il codice può essere scritto in modi diversi** mantenendo lo stesso significato. Questa è una caratteristica generale: il codice può essere scritto in molti modi diversi, equivalenti.

```
if eta < 18:
    prezzo = 5
elif studente:
    prezzo = 7
else:
    prezzo = 10
```

3 Altre considerazioni

3.1 Altre considerazioni: Condizioni mutuamente esclusive

Le condizioni rappresentano casi alternativi. Se una condizione è vera, tutte le altre devono essere false. È la situazione tipica della **scelta multipla**.

Nel caso di condizioni mutuamente esclusive è utile considerare l'**azione di default**. Quando nessuna condizione prevista è soddisfatta, esegui l'azione di default. Serve spesso per **gestire errori o input inattesi**.

```
if cond_1:
    # azione 1
elif cond_2:
    # azione 2
elif cond_3:
    # azione 3
else:
    # azione di default
```

Concretamente:

```
if op == "+":
    r = a + b
elif op == "-":
    r = a - b
elif op == "*":
    r = a * b
else:
    print("operatore non valido")
```

3.2 Altre considerazioni: Condizioni indipendenti

Ogni condizione può essere vera o falsa indipendentemente dalle altre. Più condizioni possono risultare vere nello stesso momento. In questo caso la struttura usata per la scelta multipla non è adeguata.

Esempio.

Dato un intero `a`, dire se è divisibile per i divisori $2, 3, 5$

```
a = int(input())
if a % 2 == 0:
    print('divisibile per 2')
if a % 3 == 0:
    print('divisibile per 3')
if a % 5 == 0:
    print('divisibile per 5')
```

Poiché gli `if` sono separati (non collegati da `elif`), *tutte le condizioni vengono sempre valutate*: è proprio ciò che serve, perché le condizioni sono indipendenti tra loro.

Si noti la differenza rispetto al caso di **scelta multipla**.

Il caso di scelta multipla può essere affrontato anche mediante `if` indipendenti, ma la soluzione sarebbe subottima (perché?).

```
if op == "+":
    r = a + b
if op == "-":
    r = a - b
if op == "*":
    r = a * b
```

Tornando all'esempio iniziale, come possiamo procedere se vogliamo che il programma segnali anche il caso in cui il numero non è divisibile per nessuno dei divisori considerati?

Idealmente, avremmo bisogno di un `else` collegato a tutti gli `if` precedenti. Tuttavia, nella sintassi di Python questo non è possibile: l'`else` può essere associato solo all'`if` (o all'`elif`) immediatamente precedente e allo stesso livello di indentazione.

Occorre quindi adottare una strategia diversa per rilevare il caso in cui nessuna delle condizioni risulta vera.

3.3 Variabile *flag*

Come posso controllare se **almeno una delle condizioni** è vera, o viceversa se nessuna delle condizioni è vera?

Uso una **variabile flag**, o variabile di controllo, che vale `False` prima della struttura condizionale, e viene posta a `True` se almeno una condizione è vera.

```
divisibile = False # flag
a = int(input())
if a % 2 == 0:
    divisibile = True
    print('divisibile per 2')
if a % 3 == 0:
    divisibile = True
    print('divisibile per 3')
if a % 5 == 0:
    divisibile = True
    print('divisibile per 5')

if not divisibile:
    print('Non è divisibile per 2, 3 o 5')
```

3.4 Variabile contatore

In alcuni programmi è utile **contare quante volte si verifica una certa condizione**. Per farlo si utilizza una variabile contatore, cioè una variabile che viene inizializzata a 0 e che viene incrementata ogni volta che si verifica un certo evento.

Nel seguente esempio vogliamo verificare se un numero intero è divisibile per 2, 3 o 5. Il programma deve anche indicare quanti di questi divisori sono stati trovati.

```
a = int(input())

n_divisori = 0 # VARIABILE CONTATORE

if a % 2 == 0:
    print('divisibile per 2')
    n_divisori = n_divisori + 1

if a % 3 == 0:
    print('divisibile per 3')
    n_divisori = n_divisori + 1

if a % 5 == 0:
    print('divisibile per 5')
    n_divisori = n_divisori + 1

if n_divisori == 0:
    print('il numero', a, 'non è divisibile per i divisori esaminati')
```

```
else:
    print('il numero', a, 'è divisibile per', n_divisori, 'dei divisori
     ↪  esaminati')
```

Ogni volta che una condizione risulta vera, la variabile viene incrementata di 1, registrando così un nuovo divisore trovato. È importante notare che i tre `if` sono indipendenti: tutte le condizioni vengono valutate, perché un numero può essere divisibile per più divisori contemporaneamente.

Posso anche eliminare l'ultimo `if-else` scrivendo semplicemente (ma il caso 0 divisori non viene più segnalato con un messaggio dedicato)

```
print('divisori trovati:', n_divisori)
```

3.5 Equivalenza tra strutture nidificate e strutture flat

Le strutture nidificate possono essere riscritte come strutture flat combinando condizioni semplici mediante operatori booleani. Questo permette talvolta di ottenere codice più compatto e leggibile, evitando livelli inutili di indentazione. Tuttavia non sempre è la scelta migliore.

Versione nidificata

```
x = int(input())

if x > 0:
    print("numero positivo")
    if x % 2 == 0:
        print("numero positivo e pari")
```

Versione flat

```
x = int(input())

if x > 0 and x % 2 == 0:
    print("numero positivo e pari")
```

La prima versione consente di valutare **separatamente** le due condizioni, rendendo esplicito se è vera solo la prima o lo sono entrambe. Questa impostazione è utile quando si desidera **distinguere i diversi casi**, oppure quando si prevede che in seguito possa essere necessario gestirli in modo differente.

La seconda versione è **più compatta**, ma le due condizioni sono **combinate in un'unica espressione logica**; di conseguenza non è più possibile trattarle separatamente senza **modificare la struttura del codice**.

Si noti inoltre che le due condizioni non vengono valutate in modo simmetrico: **la prima condizione ha priorità nella valutazione**. Nella versione con `if` nidificati questa priorità è **esplicita**, poiché la seconda condizione viene controllata solo se la prima risulta vera. Nella versione *flat*, invece, la priorità è **implicita** ed è determinata dalla **short-circuit evaluation** dell'operatore `and`: la seconda condizione viene valutata solo se la prima è vera.

Altro esempio

Vogliamo classificare un intero `x` in questo modo:

- positivo pari
- positivo dispari
- negativo pari
- negativo dispari
- zero

Struttura nidificata

```
x = int(input())

if x > 0:
    if x % 2 == 0:
        print("positivo pari")
    else:
        print("positivo dispari")
elif x < 0:
    if x % 2 == 0:
        print("negativo pari")
    else:
        print("negativo dispari")
else:
    print("zero")
```

Struttura flat

```
x = int(input())

if x > 0 and x % 2 == 0:
    print("positivo pari")
elif x > 0 and x % 2 != 0:
    print("positivo dispari")
elif x < 0 and x % 2 == 0:
    print("negativo pari")
elif x < 0 and x % 2 != 0:
    print("negativo dispari")
else:
    print("zero")
```

Struttura flat + DRY (*Don't Repeat Yourself*: le condizioni ripetute vengono calcolate una sola volta e salvate in variabili)

```
x = int(input())

positivo = x > 0
negativo = x < 0
pari = x % 2 == 0

if positivo and pari:
    print("positivo pari")
```

```
elif positivo and not pari:
    print("positivo dispari")
elif negativo and pari:
    print("negativo pari")
elif negativo and not pari:
    print("negativo dispari")
else:
    print("zero")
```

Attenzione: se poniamo `negativo = not positivo` stiamo erroneamente trascurando il caso `x == 0`

La versione nidificata mette maggiormente in evidenza la struttura logica del problema; la versione flat rende invece esplicite tutte le combinazioni possibili.

Se esaminiamo le condizioni in un ordine diverso otteniamo una struttura differente, ma equivalente dal punto di vista funzionale.

```
x = int(input())

positivo = x > 0
pari = x % 2 == 0

if x == 0:
    print("zero")
elif positivo and pari:
    print("positivo pari")
elif positivo and not pari:
    print("positivo dispari")
elif pari:
    # se arrivo qui il numero è sicuramente negativo
    print("negativo pari")
else:
    # se arrivo qui il numero è sicuramente negativo e dispari
    print("negativo dispari")
```

Si noti che in una catena `if-elif-else` (**chained conditionals**) conta l'**ordine** con cui vengono valutate le condizioni, e viene eseguito solo il primo ramo la cui condizione risulta vera.

Non è detto che questa versione 'semplificata' sia migliore. Le versioni qui illustrate sono funzionalmente equivalenti (producono lo stesso risultato), ma differiscono per leggibilità.

Consideriamo il caso di un numero negativo e pari. Nella prima versione flat+DRY la condizione è esplicita

```
elif negativo and pari:
    print("negativo pari")
```

mentre nella seconda versione la condizione `negativo` è implicita, perché è determinata dalla posizione del ramo nella catena `if-elif-else`.

```
elif pari:
    # se arrivo qui il numero è sicuramente negativo
    print("negativo pari")
```

La **leggibilità del codice** è un aspetto fondamentale della programmazione. Un programma non deve essere comprensibile solo al computer, ma anche alle persone che lo leggono. Una buona leggibilità facilita la **comprensione della logica del programma**, semplifica le attività di **debug** e riduce il rischio di errori quando il codice viene **modificato o esteso**. Per questo motivo, tra soluzioni funzionalmente equivalenti, è spesso preferibile scegliere quella che rende **più chiara la logica del programma**, anche se richiede qualche riga in più o ripete alcune condizioni.

4 Esempi ed esercizi (DRAFT)

4.1 Esempio: autorizzazione all'accesso a un'area riservata

Un utente può accedere solo se:

- È loggato.
- Ha il ruolo di admin oppure è un utente premium.
- Il suo account è attivo.

```
utente_loggato = True
ruolo_admin = False
utente_premium = True
account_attivo = True

if utente_loggato:
    if account_attivo:
        if ruolo_admin:
            print("Accesso consentito - sei admin")
        elif utente_premium:
            print("Accesso consentito - sei premium")
        else:
            print("Accesso negato: permessi insufficienti")
    else:
        print("Accesso negato: account non attivo")
else:
    print("Accesso negato: utente non loggato")
```

- L'indentazione stabilisce a quale `if` si riferiscono gli `elif` ed `else` successivi

Versione con condizioni booleane.

La logica viene compressa in un'unica espressione booleana. Se l'espressione complessiva è vera, l'accesso è consentito, altrimenti l'accesso è negato.

```
utente_loggato = True
ruolo_admin = False
utente_premium = True
```

```
account_attivo = True

if utente_loggato and (ruolo_admin or utente_premium) and account_attivo:
    print("Accesso consentito")
else:
    print("Accesso negato")
```

4.2 Nidificata o flat?

Si usa una **struttura nidificata** quando:

- abbiamo bisogno di messaggi di errore più specifici.
- le condizioni sono logicamente dipendenti tra loro.
- i controlli successivi hanno senso solo se i precedenti sono superati, e vogliamo che questo sia evidente nella struttura del codice.

Si usa una **struttura flat** quando:

- vogliamo scrivere codice più chiaro e compatto.
- le condizioni possono essere espresse direttamente in un'unica espressione booleana.
- non è necessario distinguere perché la condizione è stata valutata `False`. Nell'esempio, ci interessa sapere per quale motivo l'accesso è stato negato?

Il miglior approccio dipende dal contesto e dalle esigenze di chiarezza e manutenzione.

È possibile usare un mix fra i due approcci, per esempio

```
if utente_loggato:
    if account_attivo:
        if ruolo_admin or utente_premium:
            print("Accesso consentito")
        else:
            print("Accesso negato: permessi insufficienti")
    else:
        print("Accesso negato: account non attivo")
else:
    print("Accesso negato: utente non loggato")
```

4.3 Esercizi

- Scrivere un programma che chieda all'utente 3 numeri *diversi* e dica qual è il più grande e qual è il più piccolo
- Scrivere un programma che chieda all'utente 3 numeri e dica qual è il più grande e qual è il più piccolo. Dica inoltre se sono tutti diversi, tutti uguali, due uguali.
- Scrivere un programma che chieda all'utente l'età e quale crimine ha commesso (0: nessun crimine; da 1 a 10: crimini via via più gravi). Ipotizziamo che l'utente abbia commesso al massimo un crimine. In base a queste informazioni decidere se l'utente può conseguire la patente di guida. Definire l'età minima accettabile e il livello di crimine al di sopra del

quale la patente non è rilasciata. L'esercizio si può complicare inserendo due tipi di patente (es 16 anni e 18 anni).

Attenzione, abbiamo visto che le strutture condizionali possono diventare molto complesse. Non ha senso andare oltre un certo grado di complessità (`if` di `if` di `if` di `if`…). Vedremo in seguito altri strumenti

- Scrivere un programma che accetta 3 numeri e verifica se i 3 numeri possono essere i lati di un triangolo. In caso affermativo, il programma dica se il triangolo è isoscele, equilatero, scaleno.
- Scrivere un programma che chieda all'utente i coefficienti di un'equazione di secondo grado nella forma $ax^2 + bx + c = 0$ e trovi le radici. Le radici possono essere anche complesse.

4.4 Sistema di prenotazione biglietti

Un cliente può acquistare un biglietto solo se:

- Ha almeno 18 anni oppure ha almeno 14 anni (ma meno di 18) ed è accompagnato da un adulto.
- Ha abbastanza saldo nel conto per pagare il biglietto.
- Ci sono ancora posti disponibili.

Bozza di soluzione con if nidificati

```
limite_eta_1 = 14
limite_eta_2 = 18


if eta >= limite_eta_2 or (eta >= limite_eta_1 and accompagnato):
    # Controllo età e accompagnamento

    if saldo >= prezzo_biglietto:  # Controllo saldo
        if posti_disponibili > 0:  # Controllo disponibilità posti
            print("Prenotazione confermata!")
        else:
            print("Prenotazione negata: posti esauriti")
    else:
        print("Prenotazione negata: saldo insufficiente")
else:
    print("Prenotazione negata: età non idonea")
```

Bozza di soluzione con condizioni booleane

```
if ((eta >= limite_eta_2 or (eta >= limite_eta_1 and accompagnato))
        and saldo >= prezzo_biglietto and posti_disponibili > 0):
    print("Prenotazione confermata!")
else:
    print("Prenotazione negata")
```

o meglio ancora

```
condizione_eta_accompagnamento = (eta >= limite_eta_2 or (eta >=
 ↪  limite_eta_1 and accompagnato))
condizione_economica = (saldo >= prezzo_biglietto)
condizione_posti = posti_disponibili > 0

if condizione_eta_accompagnamento and condizione_economica and
 ↪  condizione_posti:
    print("Prenotazione confermata!")
else:
    print("Prenotazione negata")
```

4.5 Prestito bancario

Un software bancario determina se approvare un prestito in base a vari criteri.

Il cliente deve avere almeno 18 anni. Se ha almeno 700 punti di credito, il prestito è approvato immediatamente. Se ha tra 600 e 699 punti, viene valutato il reddito mensile: se è superiore a 3000€, il prestito è approvato, altrimenti viene rifiutato. Se ha meno di 600 punti, il prestito è rifiutato.

4.6 Gestione ordini

Un sistema di gestione ordini di un negozio online determina i tempi di consegna stimati in base alla località dell'utente e al tipo di spedizione scelta.

Regole:

- Italia: spedizione standard 5 giorni, spedizione express 2 giorni
- Europa: standard 7 giorni, express 3 giorni
- Resto del mondo: standard 15 giorni, express 7 giorni

Bozza di soluzione.

Si noti l'utilizzo delle variabili flag `errore_spedizione` e `errore_destinazione`.

Provate a riformulare la soluzione usando variabili flag con significato opposto, es `spedizione_corretta`.

Provate a riformulare la soluzione usando altre forme della struttura condizionale.

```
destinazione = "Europa"
spedizione = "Express"

errore_spedizione = False
errore_destinazione = False

if destinazione == "Italia":
    if spedizione == "Standard":
        giorni = 5
    elif spedizione == "Express":
        giorni = 2
    else:
```

```
        errore_spedizione = True  # Opzione non valida
elif destinazione == "Europa":
    if spedizione == "Standard":
        giorni = 7
    elif spedizione == "Express":
        giorni = 3
    else:
        errore_spedizione = True
elif destinazione == "Resto del mondo":
    if spedizione == "Standard":
        giorni = 15
    elif spedizione == "Express":
        giorni = 7
    else:
        errore_spedizione = True
else:
    errore_destinazione = True  # Destinazione sconosciuta

if not (errore_spedizione or errore_destinazione):
    print("Tempi di consegna stimati:", giorni, "giorni.")
else:
    print("Errore: destinazione o tipo di spedizione non valido.")
```

4.7 Esempio: voto in base al punteggio

Determinare il voto di uno studente in base al punteggio.

Se punteggio è almeno 90 → stampa “Ottimo”.

Altrimenti, se è almeno 75 → stampa “Buono”.

Altrimenti, se è almeno 60 → stampa “Sufficiente”.

Se nessuna condizione è soddisfatta → stampa “Insufficiente”.

```
punteggio = int(input())

if punteggio >= 90:
    print("Ottimo")
elif punteggio >= 75:
    print("Buono")
elif punteggio >= 60:
    print("Sufficiente")
else:
    print("Insufficiente")
```

4.8 Esempio: promosso, rimandato o bocciato

Programma che valuta un voto e classifica l’esito tra promosso con lode, promosso, rimandato e bocciato.

```python
print("Inserisci il voto (0-100):")
voto = int(input())

if voto >= 60:
    if voto >= 90:
        print("Promosso con lode!")
    else:
        print("Promosso")
elif voto >= 40:
    print("Rimandato")
else:
    print("Bocciato")
```